Shah G Tech shop & books

Technical Report

The goal, the engineering decisions and why the alternatives were rejected, the architecture, and the security posture.

PREPARED FOR	Shah G Tech — computer sales, service and workshop
SYSTEM	Multi-tenant accounting, point of sale and workshop management
BASE CURRENCY	Malaysian Ringgit (MYR)
DATE	4 August 2026
SOURCE REVISION	80bb116
STATUS	Built and verified; not yet deployed to a server

This document describes a system that has been built and tested. Where something is not built, or is blocked on information only an outside party can supply, it is named as such rather than described as if it existed.

Contents

SECTION 1

Executive summary

Shah G Tech shop & books is a single system that runs the shop and keeps its books at the same time. Ringing a sale at the counter, booking a repair job, paying a supplier and producing a profit-and-loss statement are not separate products bolted together here — they are one ledger, written once, with the accounting consequence of every shop action posted in the same database transaction as the action itself.

That single decision is what the rest of the system is built around. A till that posts to the books later, or a set of books that is reconciled to the till at month end, will disagree with reality at some point; when it does, somebody has to work out which of the two is right. Here the question cannot arise, because there is no second record to disagree with.

162	92	1,499	95k
HTTP ENDPOINTS	DATABASE TABLES	AUTOMATED TESTS	LINES OF CODE

What exists today

The system is complete enough to run the shop: users and roles, the double-entry ledger, sales invoices and receipts, supplier bills and approvals, credit and debit notes, stock with weighted-average cost, point of sale, serial-number tracking, repair jobs, bank import and reconciliation, collections and payment links, statutory tax handling, the full set of financial statements with CSV export, period lock and year-end close, opening balances, an append-only audit trail, and an in-app assistant. There is a web interface for all of it, and a browser test that drives a real server and a real database through registration, onboarding, a cash sale and the day's takings.

It has not been deployed. The hosting kit is written and the runbook exists, but no server has been provisioned, so nothing is running anywhere yet. Section 7 sets out what that needs.

> **The one-sentence summary**
>
> The books are correct by construction rather than by discipline: the ledger cannot be edited, money is never a floating-point number, every tenant's data is separated by the database itself rather than by application code remembering to filter, and 1,382 automated tests prove those properties still hold on every change.

SECTION 2

The goal

2.1 The problem

A five-person computer shop turning over roughly RM 50–60,000 a month has an awkward set of needs. It is too small to justify an accountant on staff and too active to keep books in a spreadsheet. It sells over the counter, takes in repair jobs that may sit for weeks between quotation and collection, holds stock whose cost changes with every purchase, buys on credit from suppliers, and is subject to Malaysian tax rules that are themselves in the middle of a transition to mandatory electronic invoicing.

The off-the-shelf answers each solve part of it. General accounting products keep books well but do not run a till or a workshop. Point-of-sale products run the counter but export a summary to the accounting product afterwards, which is exactly the second record that later has to be reconciled. Neither category is built around Malaysian requirements: SST, LHDN's e-Invoice regime, FPX and DuitNow payment rails, and the ringgit as the base currency.

2.2 What this system is for

- **Run the shop.** Ring a sale, book a repair, adjust stock, take a payment — the daily work, on an iPad at the counter.
- **Keep the books as a consequence.** Every one of those actions posts its own double-entry journal in the same transaction. There is no export step, no month-end reconciliation between two systems, and no opportunity for them to diverge.
- **Be defensible to an outsider.** An auditor, a bank or LHDN asking "show me why this number is what it is" can be answered: every posting is traceable to the action that caused it, every change is in a tamper-evident audit log, and the ledger cannot be quietly edited.
- **Be honest about Malaysian rules.** Where a statutory rate or a filing field could not be verified against LHDN, RMCD or PayNet, the system ships the gap explicitly rather than a plausible-looking guess. A wrong tax rate that looks right is worse than an obvious blank.
- **Serve more than one business.** The design is multi-tenant from the database upward, so one installation can serve several shops — or a freelance accountant serving several clients — without their data ever touching.

2.3 What "finished" means for a change here

Every change is held to the same bar before it counts as done. This is written into the repository's working agreements, not left to memory:

GATE	WHAT IT MEANS
Type checks pass	The whole workspace compiles under TypeScript's strict mode.
Unit tests pass	Including property-based tests that generate thousands of random transaction sets.
Integration tests pass	Against a real PostgreSQL 16, not a mock or an in-memory substitute.
Ledger invariants hold	All fourteen accounting properties in section 4.6 still true.
Audit row written	The mutation leaves a trace naming who did it, when, and from where.
API document updated	Regenerated from the code itself, so it cannot describe a system that is not there.

SECTION 3

Engineering decisions

This section is the "why this and not that". Each choice is judged against the same four criteria, in this order: correctness for money; how badly it hurts to change later; operational cost at the price point a small shop can pay; and the practical reality of finding someone in Malaysia who can maintain it.

3.1 The database: PostgreSQL 16

This is the decision everything else leans on, and it was settled by one feature.

CONSIDERED	VERDICT	REASONING
PostgreSQL 16	**CHOSEN**	Row-Level Security lets the database itself refuse to return another tenant's rows, so isolation does not depend on every query remembering a WHERE clause. Exact NUMERIC decimals, deferred constraint triggers to enforce balanced entries, advisory locks for gapless document numbering, JSONB for audit differences, and point-in-time recovery.
MySQL / MariaDB	REJECTED	No comparable row-level security. Tenant isolation would have to be enforced by application code on every single query, where one forgotten filter is a data breach. That alone settled it; the weaker constraint-trigger story compounds it.
SQLite	REJECTED	Excellent for a single-user desktop tool. No row-level security, weak concurrent writes, and no realistic path to serving several businesses from one installation.
MongoDB / document stores	REJECTED	Double-entry bookkeeping is relational by nature: an entry has lines, lines reference accounts, and the whole thing must balance atomically. Without multi-document transactions as the default and without exact decimals, this is the wrong shape of tool for money.
Cloud-only databases	REJECTED	A shop's books should not become unreachable because a vendor changes terms. PostgreSQL runs on a RM 40/month virtual server, on a machine in the back office, or on a managed cloud service — the same system, the owner's choice.

> **Why row-level security is worth the whole decision**
>
> Isolation enforced in application code fails open: forget one WHERE clause in one query out of 162 endpoints and one business sees another's figures. Enforced by the database it fails closed — a query with no tenant context set returns nothing at all. All 74 tables holding tenant data have this switched on and FORCED (so it applies even to the table's owner), and a test asserts it for every table in the schema on every run.

3.2 Talking to the database: hand-written SQL, not an ORM

The original plan named an ORM. It was replaced during the build, and this is the one reversal worth explaining in full, because "we wrote the SQL ourselves" usually deserves scepticism.

CONSIDERED	VERDICT	REASONING
Raw SQL via postgres.js	**CHOSEN**	Tagged-template queries are parameterised by construction, so SQL injection is structurally prevented rather than guarded against. Crucially, the query the database runs is the query in the file — which is what you want visible when row-level security, composite tenant keys and exact decimals are all in play.
Prisma	REJECTED	Awkward handling of session variables, which is precisely the mechanism row-level security depends on (SET LOCAL app.tenant_id). Falling back to raw queries for the security-critical paths would mean carrying an ORM for the easy half of the work only.

CONSIDERED	VERDICT	REASONING
Drizzle	REJECTED	The original choice, and reversed on contact with the schema: composite (tenant_id, id) primary keys, deferred constraint triggers and NUMERIC(19,4) all sit at the edge of what it expresses cleanly. The abstraction was costing more than it returned.
TypeORM / Sequelize	REJECTED	Heavier abstractions with a long history of generating surprising queries. Surprise is acceptable in a content site and not acceptable in a ledger.

The cost of this choice is honest: there is more SQL to read, and no compiler checks a column name inside a query string. It is paid for by 565 database tests that run against a real PostgreSQL, which catch exactly that class of mistake — and did catch two of them during the security work described in section 5.

3.3 Money

Money is never a JavaScript number anywhere in this system, and never a floating-point value in the database. This is not a stylistic preference. In binary floating point 0.1 + 0.2 is 0.30000000000000004, and a system that adds thousands of prices a day will produce a trial balance that is out by a few sen for reasons nobody can trace.

LAYER	REPRESENTATION
Application code	A Money value object holding whole minor units (sen) as a big integer, carrying its currency. Adding MYR to USD is refused by the type system, not caught at runtime.
Database	NUMERIC(19,4) — exact decimal arithmetic. There are 140 numeric columns in the schema and 0 floating-point columns.
Web interface	Amounts are strings from end to end. The browser performs no arithmetic on money at all; the moment a screen would need to calculate, the calculation belongs on the server.
Display	RM with thousands separators; dates DD/MM/YYYY in the Asia/Kuala_Lumpur timezone.

3.4 Language and runtime: TypeScript on Node.js 22

CONSIDERED	VERDICT	REASONING
TypeScript / Node.js 22	**CHOSEN**	One language across the browser, the API and the background worker means the money type, the validation rules and the invoice shape are defined once and shared, not re-implemented three times and left to drift. The workload is database and network bound, not computational, which suits Node well. Largest pool of maintainers in Malaysia.
Java / Spring Boot	REJECTED	A genuinely strong alternative with better native decimal handling and deep enterprise-finance precedent. Rejected on team size, not on merit: it would mean two languages, two validation layers and a heavier operational footprint for a system this size. The right answer for a team of fifteen with JVM depth.
C# / .NET	REJECTED	Same reasoning as Java. Excellent decimal type; wrong fit for a one-language, small-team build.
PHP / Laravel	REJECTED	Large hiring pool locally, and the reason it was still rejected is the money handling: correct decimal arithmetic is available but opt-in, and the ecosystem's defaults do not push you toward it.

CONSIDERED	VERDICT	REASONING
Go	REJECTED	Very good for the background worker and a sensible target if the matching engine is ever extracted. Not worth a second language today.
Python / Django	REJECTED	Strong Decimal type, but a weaker shared-type story with the browser and a less natural fit for the strongly-typed contract this system leans on.

3.5 The API

DECISION	REASONING
NestJS on Fastify	Its module boundaries map cleanly onto the system's own modules, and its guard and interceptor chain gives the tenant, permission and idempotency checks one obvious home that every route passes through, rather than each route remembering to call them.
REST, not GraphQL	GraphQL was rejected on cost control and authorisation. A carelessly shaped query can scan an entire ledger, and per-field permission checks over financial data are easy to get subtly wrong. REST endpoints have a fixed, reviewable cost and one permission each.
The API document is generated	The OpenAPI specification is produced by reflecting over the same route and permission metadata the application actually dispatches on. It therefore cannot describe an endpoint that does not exist, or claim a permission the code does not enforce. A conformance test fails the build on any route that is served but undocumented, documented but unserved, or accepts a body with no validation schema.
Validation at every boundary	Zod schemas validate untrusted input at the single point where it becomes typed data, and the same schema is what the generated documentation advertises.

3.6 The web interface

Next.js 15 with React 19. What is more interesting is what was left out: the interface has five runtime dependencies in total. There is no component kit, no chart library, no state-management framework, no form library.

That was a deliberate trade. Accounting screens are dense and unusual — editable invoice grids, reconciliation workspaces, a point-of-sale keypad — and general-purpose component kits tend to be fought rather than used for those. The charts are drawn directly at true pixel geometry because a charting library that treats money as a float defeats the purpose of every other decision in this document. The cost is that some primitives are written by hand; the benefit is a small dependency surface to audit and no library standing between the screen and the numbers.

3.7 Background work: the database, not Redis

Scheduled work — payment reminders, the nightly ledger-integrity check, the weekly digest — runs from a job table in PostgreSQL, claimed with SELECT ... FOR UPDATE SKIP LOCKED. The original plan called for Redis and BullMQ, and that was reversed for a specific reason.

> **The dual-write problem**
>
> A job queued in Redis at the moment a ledger entry is written in PostgreSQL is two writes to two systems that can fail independently. The invoice posts and the reminder is lost, or the reminder fires for an invoice that was rolled back. Keeping the job in the same database as the ledger effect means they commit together or not at all — which is the entire reason the pattern exists. Introducing a second datastore to hold the job would reintroduce precisely the inconsistency it was adopted to prevent. Redis earns its place when something needs cross-process rate limiting or fan-out beyond a single poller; nothing does yet.

3.8 Smaller choices, briefly

NEED	CHOSEN	INSTEAD OF
PDF documents	pdfkit	Headless Chromium, which would mean shipping a browser in the container to render an invoice — a large attack surface and a slow cold start for a one-page document. (This report was produced by the same renderer.)
Passwords	Argon2id	bcrypt — still respectable, but Argon2id is the current password-hashing standard and resists GPU attack better.
Session tokens	Random 256-bit tokens, stored hashed	Storing tokens as issued, which turns a database backup into a set of live credentials.
Access tokens	JWT with the algorithm pinned	Accepting whatever algorithm the token declares — the classic "alg: none" forgery.
Integration testing	A real PostgreSQL 16	Testcontainers (needs Docker, which is not available in every environment this is developed in — a test harness that cannot start is one nobody runs) and mocks (which prove the mock works).
Ledger testing	Property-based tests	Only hand-written examples, which check the cases you thought of. These generate thousands of random transaction sets and assert the accounting laws hold across all of them.

3.9 Where the built system departs from the original plan

Four decisions were reversed on contact with the actual work. They are recorded here because a plan that was never revised is usually a plan nobody tested.

ORIGINALLY PLANNED	ACTUALLY BUILT	WHY IT CHANGED
Drizzle ORM	Raw SQL	The schema’s composite keys, session variables and deferred triggers sat at the edge of what it expressed well.
BullMQ on Redis	PostgreSQL outbox	Would have reintroduced the dual-write inconsistency the outbox exists to remove.
Testcontainers	Real PostgreSQL service	Requires Docker, which is not present in every development environment used here.
Chromium for PDFs	pdfkit	A whole browser to render one page of text is disproportionate in both size and attack surface.

SECTION 4

Architecture

4.1 The shape

It is a modular monolith: one deployable API with firm internal module boundaries, rather than a set of microservices. For a system of this size that is the correct trade. Microservices would buy independent scaling that is not needed at this volume, and would charge for it in distributed transactions across a ledger — which is the last place anyone should want them.

COMPONENT	ROLE
apps/web	The Next.js interface. Holds no arithmetic: amounts are strings end to end, and formatting is the only numeric work it does.
apps/api	The NestJS server. Authentication, permissions, and all 162 endpoints.
apps/worker	Drains the outbox and runs scheduled jobs. Connects as a separate database role with rights the internet-facing API does not have.
packages/domain	The accounting rules as pure functions — money, journal entries, the tax engine, the matching engine. Zero runtime dependencies and no input or output of any kind; this is enforced automatically.
packages/db	The schema (38 migrations), the security policies, and the repository services.
packages/contracts	Shared validation primitives so one constraint cannot mean two different things on two endpoints.

Why the domain package has no dependencies

The double-entry rules and the tax engine are pure functions over values. That makes them testable at enormous volume — the property tests generate thousands of transaction sets per run — and impossible to accidentally couple to a database session or a web request. It is checked in continuous integration: an import of anything that performs input or output fails the build.

4.2 What happens on every request

Every request passes the same chain, in this order, and the order is deliberate:

#	STAGE	PURPOSE
1	Request context	A request identifier exists before anything can fail, so any error can be traced.
2	Rate limit	The cheapest possible rejection, before any database work. Deliberately ahead of authentication: otherwise an unauthenticated flood still costs a signature verification and a query each time.
3	Authentication	Verify the token, with its signing algorithm pinned.
4	Tenant resolution	The business named in the token must match the one named in the request header. Disagreement is refused.
5	Permission check	Does this member’s role — narrowed by an API key’s scopes, if one is in use — carry the permission this endpoint requires.

#	STAGE	PURPOSE
6	Idempotency	Any write must carry a key, so a retry after a dropped connection cannot charge a customer twice.
7	Handler	Opens the database transaction, which sets the tenant for the row-level security policies.

4.3 The ledger

Three rules govern the ledger, and they are enforced by the database rather than by convention.

- **It is append-only.** A posted journal entry is never updated and never deleted. A correction is a new reversing entry that references the original. A database trigger refuses anything else, so this holds even against a direct connection.
- **Everything balances.** Debits must equal credits within every entry, checked by a deferred constraint trigger at the moment the transaction commits — not by application code that could be bypassed.
- **One way in.** All journal writes go through a single posting service. Nothing else writes the ledger tables, so there is exactly one place where the rules can be enforced and exactly one place to audit.

4.4 The data model in numbers

92	227	104	38
TABLES	INDEXES	TRIGGERS	MIGRATIONS

Of the 92 tables, 73 hold tenant data and carry the tenant identifier as the first part of their primary key. 74 tables have row-level security enabled and forced, protected by 74 policies. Migrations are hand-written SQL, applied forward only, and reviewed like production code.

4.5 The fourteen invariants

These are the accounting properties that must be true at all times. They are tested, not assumed, and several are checked by generated data rather than fixed examples.

#	PROPERTY THAT MUST ALWAYS HOLD
1	Every posted entry has debits equal to credits.
2	Across all entries, total debits equal total credits — the trial balance balances.
3	Assets equal liabilities plus equity, at every point in time.
4	Reversing every entry returns every account balance to zero.
5	The cached period balances always equal the figures recomputed from the underlying lines.
6	The receivables control account equals the sum of outstanding customer invoices.
7	The payables control account equals the sum of outstanding supplier bills.
8	The bank account balance equals its opening balance plus reconciled transactions.
9	Nothing posts into a locked period without both the override permission and a logged event.
10	The same idempotency key applied twice produces exactly one posted entry.

#	PROPERTY THAT MUST ALWAYS HOLD
11	Document numbering has no gaps within a business and document type.
12	Every invoice line's tax equals what the tax engine computes for that invoice's tax point date.
13	Settling a foreign-currency invoice at a different rate produces a balancing exchange gain or loss.
14	No query executed without a tenant context returns any tenant row.

Invariant 14 is the one that proves the isolation boundary, and it runs against every table in the schema rather than a chosen sample. Invariant 8 carries a stated precondition — it holds only when the account is fully reconciled — because on a real bank account there is almost always a payment in transit, and a check that fails constantly is a check somebody switches off.

4.6 The audit trail

Every mutation writes an audit row naming who did it, when, and from which address. The rows form a hash chain: each carries a digest of the one before it, so altering or removing a row anywhere in the history breaks the chain from that point onward and a verification function reports exactly where. The application's database role is granted no ability to update or delete audit rows, and a trigger refuses the attempt even for a role that holds the grant.

SECTION 5

Security

5.1 The approach

Security here is layered so that no single mistake is sufficient. The layers are, from the outside in: the network (only one port is published); the request chain (rate limiting before authentication, then authentication, then permissions); the application (validation at every boundary, parameterised queries throughout); and the database (row-level security, restricted roles, and triggers that refuse forbidden writes regardless of what the application asks for).

The deepest layer is the important one. An application bug — a forgotten filter, a mistyped condition — cannot leak another business's data, because the database will not return rows the current tenant context does not cover.

5.2 Tenant isolation, verified

These figures are read from a freshly migrated database rather than counted by hand:

PROPERTY	MEASURED	REQUIRED
Tables with row-level security enabled and FORCED	**74 / 74**	all tenant tables
Policies carrying an explicit write check as well as a read check	**74 / 74**	all policies
Privileged database functions with a pinned search path	**25 / 25**	all functions
Privileged functions executable by any database role	**0**	zero
Floating-point columns anywhere in the schema	**0**	zero

PROPERTY	MEASURED	REQUIRED
Identity tables reachable directly by the application role	0	zero

The last row is worth expanding. User accounts and sessions are global by design — one login serves a person across every business they work for — so a per-tenant policy cannot express their protection. Instead the application's database role is granted nothing at all on those tables. Every legitimate operation goes through a small number of narrow, individually reviewed database functions. That is a stronger guarantee than a policy, and it is asserted by a test.

5.3 Identity and access

CONTROL	IMPLEMENTATION
Passwords	Argon2id. A failed login for an unknown account still performs a dummy hash, so response timing does not reveal whether an email is registered.
Sessions	Random 256-bit tokens, stored only as hashes and compared in constant time. Rotated on use; reusing a spent token revokes the whole session family, which is what detects a stolen token.
Access tokens	Short-lived JWTs with the signing algorithm pinned. The business the token names must match the one the request header names.
Roles	Nine ranked roles from Owner down to External Auditor. Nobody may grant a role above their own, and — since the security review — nobody may act on a member who outranks them.
API keys	Scoped, and the scopes are intersected with the role's permissions rather than added to them. A key can only ever be narrower than the person who issued it.
Unknown records	Asking for another business's record returns "not found", never "forbidden". A forbidden response would confirm the record exists.

5.4 The security review

A full adversarial review was carried out against the whole surface — authentication, tenant isolation, the unauthenticated payment pages, the assistant, and the container images — deliberately attempting to defeat each. Ten findings of substance were confirmed. All ten are fixed; the fixes are covered by tests, and the full detail is in the repository's settlement register.

SEVERITY	FINDING	RESOLUTION
HIGH	The server trusted client-supplied forwarding headers, so any caller could present a false address — defeating rate limits and falsifying the address recorded in the audit trail.	Trusts nothing by default and uses the real connection address. A setting declares how many proxies genuinely sit in front, documented in the deployment runbook.
HIGH	Two endpoints without an explicit permission requirement read the full role permissions instead of an API key's narrower scopes, so a key issued for one purpose could reach the whole financial picture through the assistant.	A single function now computes the genuinely effective permissions by intersecting role and scope; the assistant uses it. Covered by a property test.
MED	An administrator could re-role an owner and strip the one person senior to them, leaving the business without an owner.	Acting on a member who outranks you is refused, independently of which role is being granted. Covered by an end-to-end test.

SEVERITY	FINDING	RESOLUTION
MED	A system-wide job table returned raw error text to any business; those errors can embed another business's data.	The error text is no longer returned. The success or failure status, which is all a tenant needs, remains.
MED	A second, distinct payment notification for an already-settled invoice could settle it twice.	A settled payment link refuses any confirmation carrying a different key, while a genuine retry of the same one still succeeds. Covered by a test.
MED	Signing out accepted a session identifier that is readable inside any access token, so it could be used to sign out somebody else.	Signing out now requires the refresh token, a secret only its holder has.
MED	The maximum session age was defined but never actually enforced.	The session family's start time is now readable and the ceiling is enforced on refresh.
MED	An account lockout could be sustained indefinitely by one wrong password per window, locking a legitimate user out permanently.	An expired lockout now clears and starts a fresh count.
MED	Some privileged database functions could be executed by any database role and did not pin their search path — a latent full-takeover the day another role is added.	A migration revokes public execution and pins the search path on every privileged function. Verified: zero remain.
MED	No browser security headers were set, and the payment webhook re-encoded the message before checking its signature — so a real provider's signature would never have matched.	Content security policy, clickjacking and transport-security headers added at both the application and the edge. The exact bytes received are preserved and verified.

Alongside these, a set of hardening measures was applied: the containers now run as an unprivileged user with all Linux capabilities dropped and privilege escalation disabled; secrets, test code and backups are excluded from the images; the assistant carries its own per-business rate cap because its cost is a paid model call rather than a database query; and the test suite now asserts structurally that every isolation policy carries an explicit write check.

5.5 Attack surface

As configured for deployment, exactly one port is published: the web interface. The API, the background worker and the database are reachable only on the private network between containers. No unintended open port was found.

The only part of the system that serves tenant data without authentication is the customer payment page, which is unavoidable — a customer following a payment link has no account. It is confined accordingly: its own much tighter rate limit; a random high-entropy token; unknown, expired and cancelled links all answer identically so guessing yields no signal; and the response carries only the amount, the reference, the invoice number and the shop's name. Payment is never confirmed by the customer's browser — only by a signed server-to-server notification from the provider — because a browser returning from a bank proves nothing about whether money moved.

What this review does not claim

This was a review by the same party that wrote the system, which is a real limitation: it is much easier to find the mistakes you did not know you were making when somebody else looks. An independent penetration test before the system holds real money is recommended in section 7, and nothing here should be read as a substitute for one.

SECTION 6

Statutory compliance

Malaysian payroll has four compulsory deductions, and every one of them is a lookup table published by a different authority. This chapter is separate from the rest because the engineering question it answers is different: everywhere else the risk is a bug, and here the risk is being confidently, plausibly, quietly wrong.

The single number that explains this chapter

The employer's EPF contribution is thirteen percent of wages up to RM 5,000, and twelve percent above it. Every summary, every forum answer and every spreadsheet template says "twelve". On a RM 2,500 wage the published schedule says RM 325 and twelve percent says RM 300 — an under-deduction of RM 25 a month that compounds silently for as long as nobody checks, and which is the employer's liability to make good, with penalty. Not the employee's.

That is why nothing in this system multiplies a wage by a percentage. All four schemes are tables in law, the tables are transcribed from the legal instruments, and the instruments themselves are committed into the repository beside the code so any figure can be traced to the page it came from. The percentages people quote are a description of these tables, not the rule.

6.1 What is carried, and from where

DEDUCTION	SOURCE INSTRUMENT	ROWS	EFFECTIVE
EPF	Employees Provident Fund Act 1991, Third Schedule — Parts A, C, E and F	**1,203**	01/10/2025
SOCSO	Employees' Social Security Act 1969 (Act 4), including SKBBK	**65**	01/06/2026
EIS	Employment Insurance System Act 2017 (Act 800)	**65**	01/10/2024
PCB	IRBM Specification for MTD Calculations using Computerized Calculation 2026	**9**	01/01/2026

None of these are constants in code. They are rows in effective-dated tables, read as at the contribution month rather than as at today, and the distinction is not academic: SKBBK — the 24-hour non-work accident scheme — began on 1 June 2026 and added an employee deduction to a SOCSO category that previously took nothing. A system reading "the current rates" silently restates every payslip it ever produced the first time a rate changes.

6.2 Three findings worth recording

Each of these was found by checking the implementation against the published source, and each would have produced figures that looked entirely reasonable.

- Secondary sources disagree with each other about it, and the reason is that they are quoting different Parts of the same Schedule without saying so. Part E covers Malaysian citizens aged 60 and over — the employer contributes and the employee does not — while Part C covers permanent residents of the same age, who still pay. The distinction is invisible in every summary and is confirmed in the data itself.
- Earlier research said an employee aged 60 or over contributed nothing. That was true and is now out of date: from 1 June 2026 SKBBK applies to both categories. Category 1 also carries two separate employee columns — Invalidity and SKBBK — which the payslip shows separately, because PERKESO's own statement does and a single combined figure cannot be reconciled against it.
- PCB projects the whole year — this month's pay is assumed to repeat, the year is taxed, what has already been deducted is subtracted, and the remainder is spread over the months remaining. The consequence is testable and counterintuitive: on the same wage in the same month, someone paid since January owes RM 207.50 and someone who started that month owes nothing, because a part year falls where the RM 400 individual rebate covers the tax.

6.3 How the figures are proved

Two independent checks, deliberately not sharing a source. The calculation engine is pure — it touches no database — which is what makes the first one possible at all.

CHECK	WHAT IT DOES
Every published band	The contribution engine is run against all 1,203 EPF bands and all 65 SOCSO and EIS bands, twice per band, reading the same transcribed tables the migration was generated from. An engine that disagrees with the schedule anywhere fails here.
The authority's own worked example	IRBM publishes a four-month calculation with its specification — January and February plain, March with optional reliefs, April with an RM 8,250 bonus. The tax engine reproduces all four exactly, month by month, including the accumulation between them.
The loader, separately	Against a real database: that the migration loaded the tables faithfully, and that a payroll dated to an earlier month gets that month's schedule. Either check alone is a half-check.
Through the browser	The end-to-end journey costs a hire and produces a payslip against the real server and database, so a regression in the schedule, the loader or the formula fails the browser test too.

> **What the tests caught, in both directions**
>
> Reproducing the authority's worked example found a genuine defect: one intermediate value must be computed differently in two steps of the bonus formula, and computing it the same way in both leaves each step individually plausible and the answer wrong. In the other direction, several of my own test expectations turned out to be wrong where the engine was right — the RM 325 above was one, and a rounding rule that truncates before it rounds up was another. Both are recorded as tests now, with the reasoning, so the next reader does not re-derive them.

6.4 What is deliberately not claimed

Two of the five tax formulae in the specification are not implemented: the Returning Expert Programme and the Knowledge Worker rate both require prior Ministerial approval that a computer shop will not hold, and there is no code path that quietly falls back to them. No year before 2026 is carried — an earlier date raises an explicit error rather than applying this year's schedule backwards through a Budget.

And the whole of this is a calculator, not a payroll run. It keeps no employee records, files nothing with any authority, and posts nothing to the accounts. That ordering was chosen: the rates had to be demonstrably right before anything built on top of them was worth having, because a pay run shipped first would have produced confident figures from day one that were wrong by a few sen on most salaries.

SECTION 7

What I would add next

This section is the honest list of what is missing, in the order I would do it. It is divided by what is stopping each item, because "not built" and "cannot be built yet" are different situations and mixing them hides the ones that need a decision from you.

7.1 Before the system holds real money

ITEM	WHY IT MATTERS
Deploy it	Nothing is running anywhere. The hosting kit and runbook exist; what is needed is a server (a RM 40–80/month virtual machine is sufficient at this size) and a domain name. Everything else here is blocked behind this.
HTTPS	Sessions over plain HTTP on shop wireless are credentials in the clear. The configuration is written and obtains its own certificate automatically; it needs the domain to exist.
Off-site backups	A nightly database dump is configured and keeps fourteen days, but it writes to the same machine. The server that fails takes its backups with it. This needs a storage bucket and roughly ten lines of configuration.
A rehearsed restore	A backup nobody has restored is a hope, not a backup. Restore into a scratch database once, deliberately, and record how long it took.
Two-factor authentication	Designed but not built. For the Owner and Administrator roles specifically, a stolen password should not be sufficient. This is the largest genuine gap in the security posture and the next thing I would build.
An independent penetration test	For the reason in the note above. Worth doing once the system is deployed and before it carries a year of real books.

7.2 Blocked on something only you or an outside party can supply

Each of these is built up to the boundary and stops where a credential or a document is required. None is blocked on engineering work.

FEATURE	WHAT UNBLOCKS IT

FEATURE	WHAT UNBLOCKS IT
e-Invoice submission to LHDN	MyInvois sandbox credentials. This is the one with a statutory deadline attached — Malaysia's e-Invoice mandate is being phased in by turnover band, and the applicable date for a business this size should be confirmed with LHDN directly rather than assumed from this document.
Online payment collection (FPX, DuitNow)	Merchant credentials from a payment provider such as Billplz or iPay88. The clearing-account accounting, the payment pages and the webhook handling are built and tested against a simulated provider; only the provider-specific client is missing, deliberately, because one written against documentation alone would look finished and probably be wrong.
DuitNow QR codes	The merchant template from PayNet. The public encoding standard is implemented and verified; the Malaysia-specific fields are not guessed, because a plausible-looking wrong code would pay the wrong party.
Live assistant replies	An API key. Without one the assistant reports itself unconfigured and the built-in guidance still works.
Emailed invoices and reminders	An email service key. The reminder engine, its schedule and its wording are built and can be run with a human pressing send.
Withholding tax rates, SST filing boxes, unit-of-measure codes	Verification against LHDN and RMCD source documents. These ship empty on purpose — an incorrect rate that looks correct is worse than a visible gap.
Bank statement import for other banks	A sample statement file from each bank. The Maybank payment-advice format is built from a real sample; guessing another bank's column layout is exactly the failure that explicit per-bank profiles exist to prevent.

7.3 Worth doing soon after launch

- **Error tracking.** Automatic reporting when something fails in production, with financial details scrubbed before they leave the machine. At present a failure is visible only in the server log, if somebody looks.
- **Structured logging and tracing.** Every log line tagged with the business and request identifier. The first question in any incident is "which business, which document, which request", and the system should be instrumented for that question specifically.
- **Shared rate limiting.** The current limiter holds its counters in memory, which is correct for one server and wrong for two. Moving it to shared storage is the point at which Redis genuinely earns its place.
- **Data retention and export.** Malaysian personal data law expects a stated retention position and a way to answer a request for a person's data. Worth writing down before it is asked for.
- **Receipt capture on a phone.** Photograph a supplier receipt at the counter, queue it, and let it upload when the signal returns. The single highest-value convenience feature for a shop of this kind.
- **A second server.** Today one machine failing takes the shop offline. Whether that matters is a business decision about how long the counter can run on paper.

7.4 Later, and only when the size demands it

- **A read replica** so heavy reports do not compete with the till. Not needed at this volume.
- **Table partitioning** for the ledger lines and audit log once they pass tens of millions of rows. The schema is already designed so this can be done without restructuring.
- **Multiple branches** if a second shop opens — stock and takings per location.
- **Payroll** with EPF, SOCSO and PCB. A substantial piece of statutory work in its own right and deliberately out of scope so far.

- **Malay and Chinese interfaces.** The interface is English today; the groundwork for translation is worth laying before there are hundreds of screens.

7.5 One thing I would change about how it is built

The rate limiter keeping its counters in memory is the clearest example of a decision that is correct today and wrong the moment there are two servers, and it is documented as such rather than left to be discovered. I would rather record that plainly here than have it found later and read as an oversight. The same is true of the assistant's per-business cap, which is enforced in the same memory and shares the same limitation.

SECTION 8

Verification

Everything asserted in this document is checked automatically. The figures below were produced by running the full suite immediately before this report was generated.

SUITE	TESTS	WHAT IT PROVES
Accounting rules (pure)	**696**	Money arithmetic, double-entry validity, tax computation, matching — including property tests over generated transaction sets.
Database	**565**	Against a real PostgreSQL 16: isolation, the invariants, append-only enforcement, the audit chain.
API (end to end)	**200**	Through the real server: authentication, permissions, every endpoint, the generated document's conformance to the router.
Background worker	**21**	Outbox draining, retries, dead-lettering, scheduled jobs.
Shared contracts	**17**	The validation primitives shared between the interface and the server.
Total	**1,499**	Across 97 test files, plus one browser test driving a real server and database through a full first day of trading.

In addition: the whole workspace type-checks under strict mode; both builds of the web interface compile; and all 38 migrations apply cleanly to an empty database, which is how the schema figures in this report were measured.

> **A note on how this document was produced**
>
> This report is generated by a script in the repository, from figures measured against the source tree and a freshly migrated database. It is not typed by hand, for the same reason the API documentation is not: a hand-written description of a system that keeps changing is wrong within weeks, and no reader can tell which paragraph went stale. Re-running the script re-measures and reproduces it.